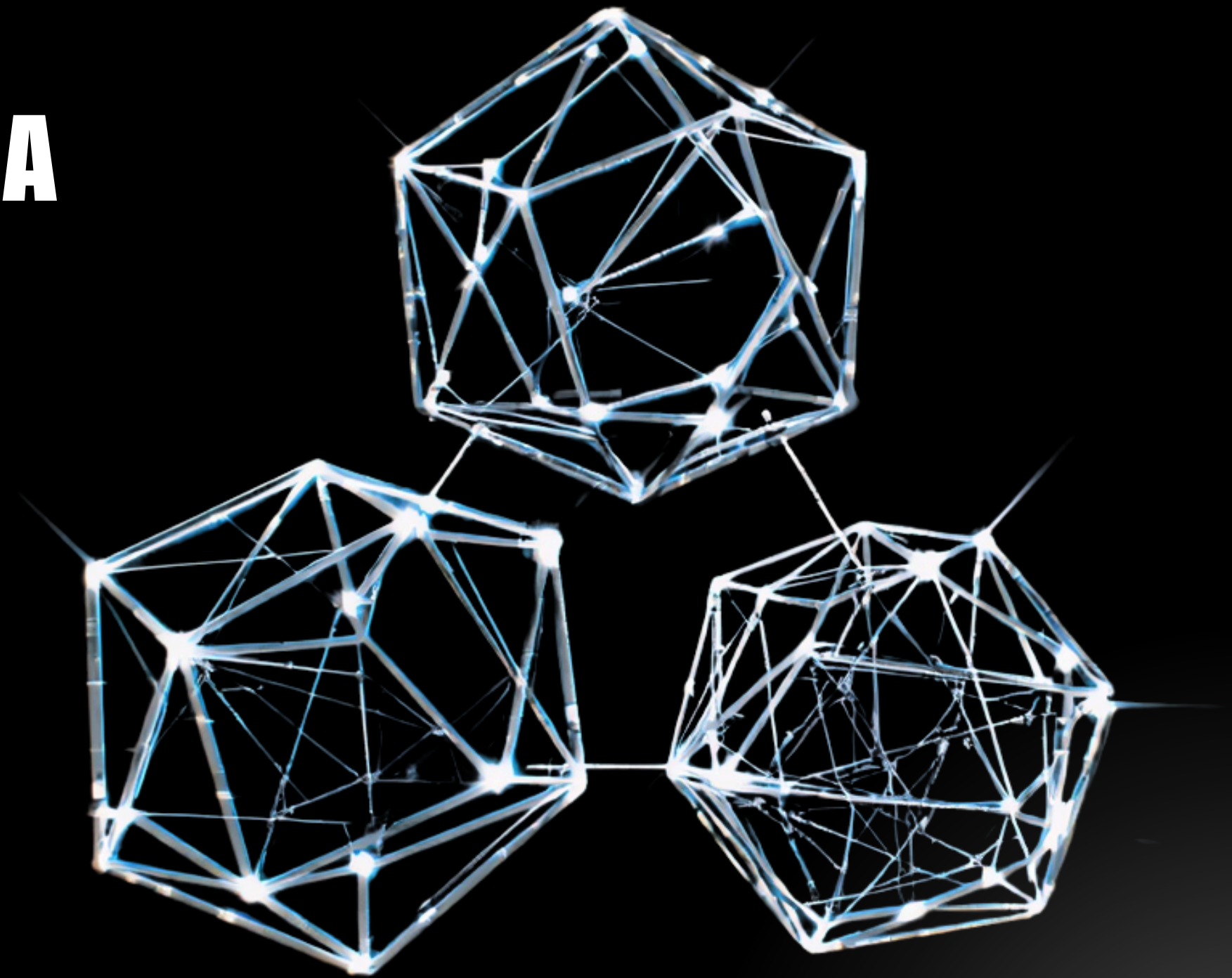


CÓDIGO LIMPO E ARQUITETURA

Dividir responsabilidades em um projeto de software é essencial para manter o código organizado, escalável e de fácil manutenção. Utilizando princípios de código limpo e boas práticas de arquitetura, cada componente do sistema passa a ter um papel bem definido, reduzindo acoplamento e aumentando a clareza do projeto.



NOSSO TIME



 github.com/Maciel64
macielsuassuna14@gmail.com
RA: 12823141083



 github.com/Juvito12
joaovictorsilvadev1@gmail.com
RA: 12823112610



 github.com/Ftauan
felipetauan11@gmail.com
RA: 12823131535



 github.com/Arturfurtadoc-lang
arturfurtadoc@hotmail.com
RA: 1282312745



 github.com/Viniciusolinto
viniolinto2345@gmail.com
12823135297

O QUE É CÓDIGO LIMPO?

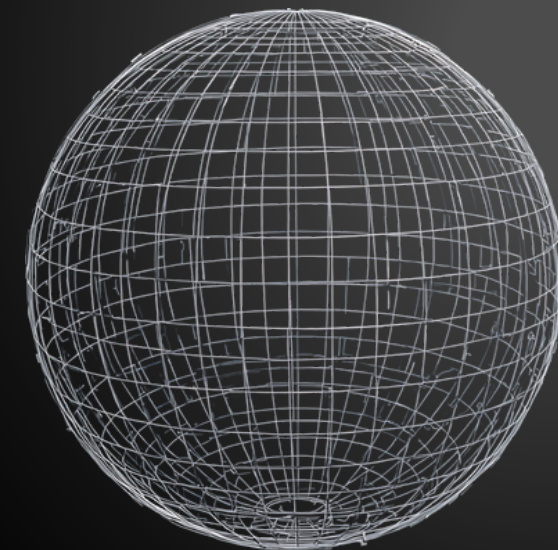
O conceito de código limpo ganhou força principalmente com a evolução da engenharia de software e foi muito popularizado por boas práticas consolidadas ao longo do tempo. A ideia central surgiu de um problema comum: sistemas que funcionam, mas são difíceis de entender, modificar e manter.

Antes disso, muitos projetos eram construídos apenas com foco em “fazer funcionar”, sem preocupação com organização. Com o crescimento dos sistemas e das equipes, ficou claro que isso não era sustentável. Nesse contexto, surgiu o conceito de Código Limpo (Clean Code), termo cunhado por Robert C. Martin, também conhecido como Uncle Bob, que defende a ideia de escrever códigos simples, legíveis e fáceis de manter.

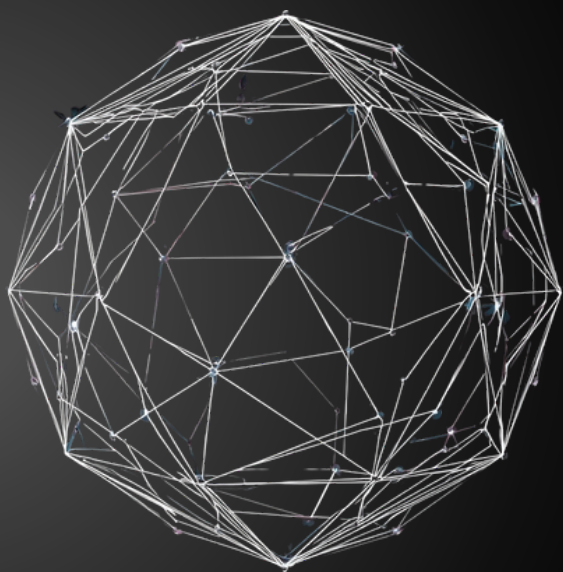
Assim, o código limpo propõe que o desenvolvimento de software vá além do funcionamento do sistema, priorizando também clareza, organização e facilidade de manutenção para outros desenvolvedores e para futuras atualizações.



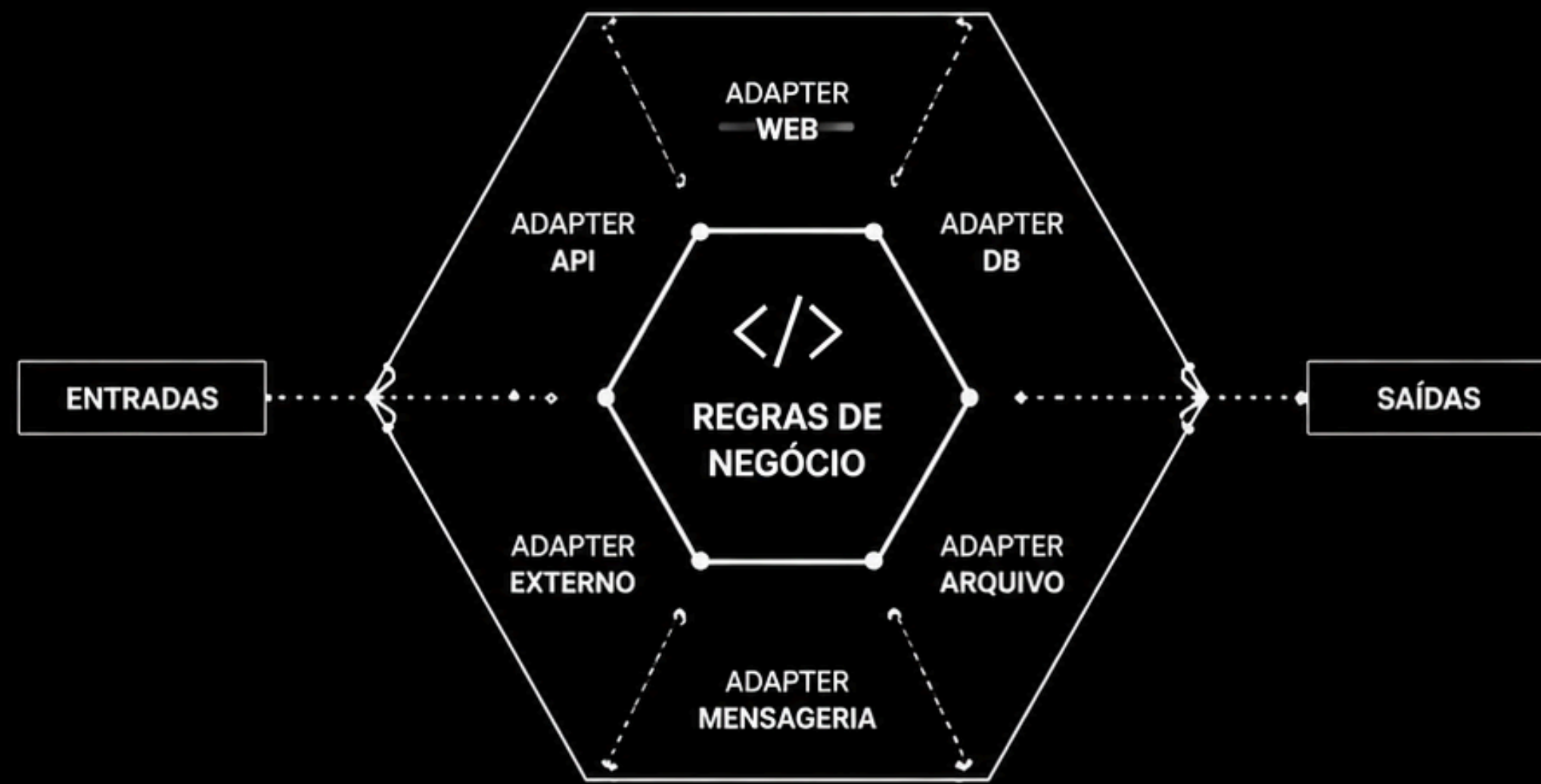
ARQUITETURA HEXAGONAL (Ports and Adapters)



A arquitetura hexagonal, também conhecida como Ports and Adapters, foi proposta por Alistair Cockburn com o objetivo de resolver um problema clássico: o forte acoplamento entre a lógica de negócio e as tecnologias externas, como banco de dados, APIs e interfaces. Antes disso, era muito comum que regras de negócio ficassem misturadas com código de infraestrutura. Isso dificultava testes, manutenção e evolução do sistema. A proposta da arquitetura hexagonal é inverter essa lógica.



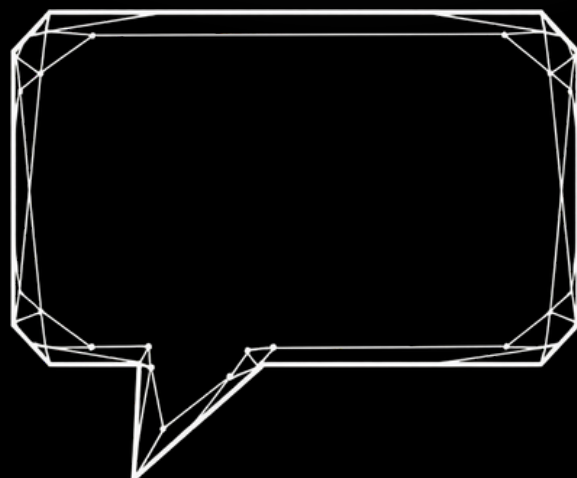
Alistair Cockburn



DEPENDÊNCIAS EXTERNAS | ISOLAÇÃO | TESTABILIDADE | FLEXIBILIDADE | EVOLUÇÃO

PRINCIPAIS IDEIAS

Controller



O Controller é responsável por receber as requisições do usuário, interpretar os dados de entrada e direcionar a execução para a camada correta. Ele atua como intermediário entre o cliente e a aplicação, sem conter regras de negócio.

Service



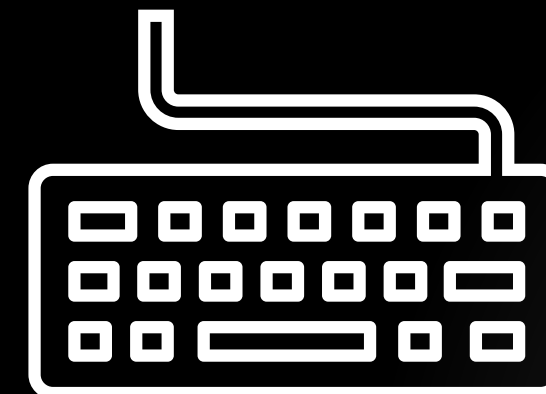
O Service concentra a lógica de negócio da aplicação, sendo responsável por processar dados, aplicar regras e coordenar operações entre diferentes partes do sistema.

Use Case



O Use Case representa uma ação específica do sistema, focando em uma única funcionalidade, como criar um usuário ou finalizar uma compra. Ele organiza a lógica de negócio de forma mais estruturada e orientada ao que o usuário realmente faz.

Repository



O Repository é responsável pelo acesso e manipulação dos dados, funcionando como uma abstração do banco de dados. Ele permite que a aplicação não dependa diretamente da tecnologia de persistência utilizada.

IMPACTO DO CÓDIGO LIMPO

Redução de Bugs

Aumento de
Produtividade

Facilidade de
Manutenção

MINICURSO CÓDIGO LIMPO

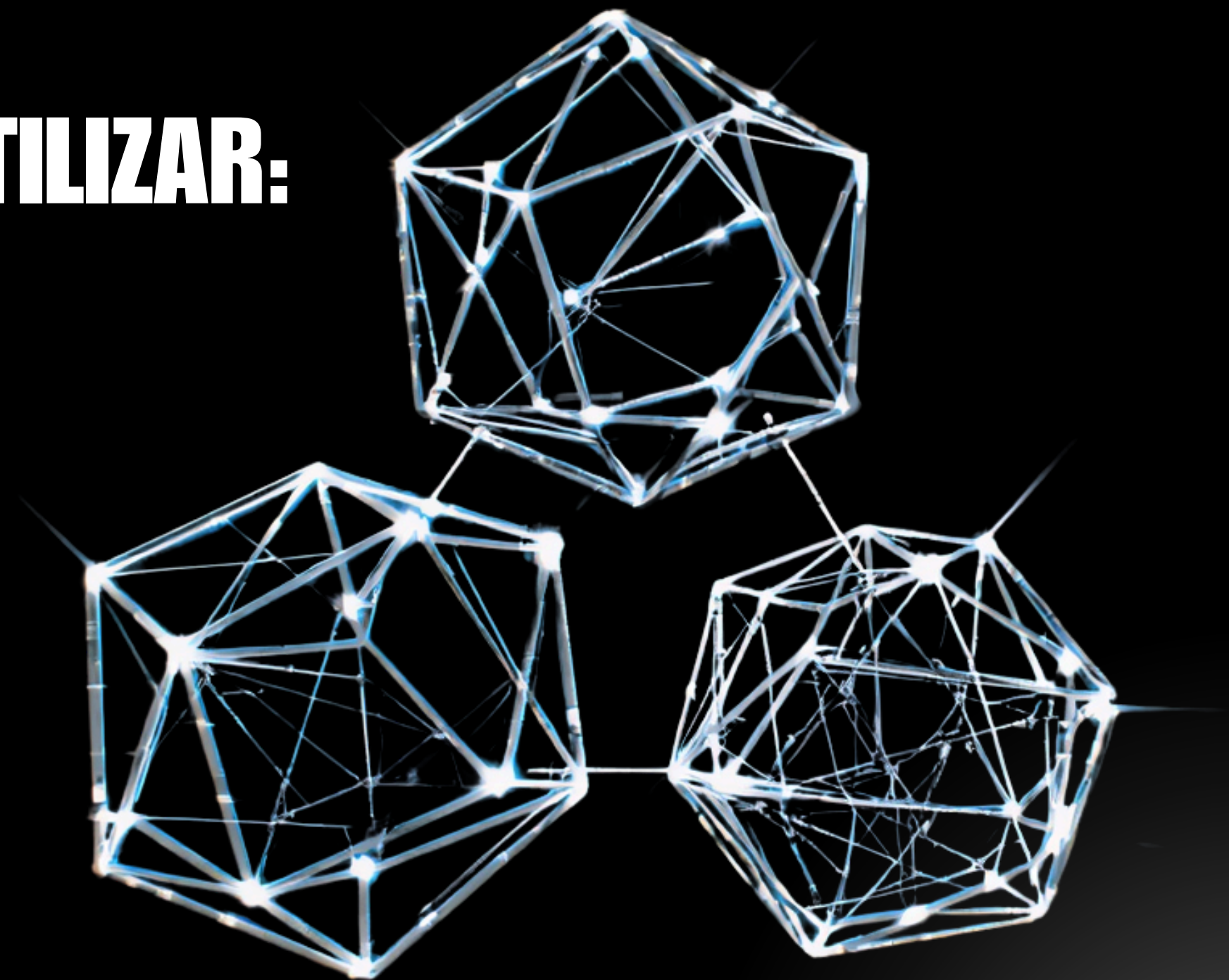


O QUE SERÁ APRESENTADO:

- O que é realmente importante:
 - Manter padrões
 - Garantir pontos de verdade únicos no seu código
 - Garantir que os fluxos principais sejam identificáveis e legíveis
 - Refatoração de um código funcional, seguindo padrões para alcançar um código mais limpo
 - Pequenas dicas de boas práticas no código, a fim de atingir um código mais legível e manutenível

FERRAMENTAS QUE VAMOS UTILIZAR:

- BACKEND –
 - BunJs
 - Elysia
 - Prisma (ORM).
 - Resend (Envios de Emails).
 - Handlebars
 - Neon (Banco de dados).
 - Asaas (Geração de pagamentos).



ACESSE O CÓDIGO DA APRESENTAÇÃO



[HTTPS://GITHUB.COM/MACIEL64/MINICURSO-UNP](https://github.com/maciel64/minicurso-unn)